**Abstract**

We describe the systems and infrastructure details of training ESM2-small, a 9.6M-parameter protein language model, on a Cambricon MLU370 accelerator. Over approximately 11 hours on a single MLU370 card, the model processes 67,500 gradient steps across 5 epochs, consuming roughly 1.1 billion tokens from Swiss-Prot at a throughput of 30,000 tokens/second. This paper details the training pipeline architecture, hardware-software co-design considerations for the MLU370 platform, and practical observations on training stability, gradient checkpointing, and mixed-precision strategies for Transformer-based protein models.

**Keywords:** protein language model, Cambricon MLU370, distributed training, masked language modeling, systems for ML

# 1 Introduction

Training large neural networks requires careful co-design of software pipelines and hardware accelerators. While GPU-based training pipelines are well-documented in the literature [**? ? ?** ], details of training on domestic Chinese accelerators such as the Cambricon MLU370 remain less publicly explored. This paper fills this gap by presenting a detailed account of the systems engineering behind training ESM2-small on MLU370.

The Cambricon MLU370 is a domain-specific accelerator for ML workloads, offering on-chip SRAM and a programming model based on CNNL (Cambricon Neural Network Library). Our work adapts the ESM-2 training codebase to MLU370, with modifications to memory management, gradient synchronization, and mixed-precision operators.

# 2 Hardware and Software Stack

## 2.1 Cambricon MLU370 Architecture

The Cambricon MLU370 series (also known as Shenlong) targets AI inference and training workloads with dedicated tensor and vector accelerator cores. Key specifications relevant to our workload:

- On-chip shared memory (SRAM) for activations and intermediate results
- Support for INT8, FP16, BF16, and FP32 compute primitives
- CNNN (Cambricon Neural Network) operator library with fused kernel implementations
- Multi-card interconnect via MLULink for gradient all-reduce in multi-worker settings

## 2.2 Software Stack

We use the following software components:

- **PyTorch 2.x** with Cambricon backend (`torch.mlu` device)
- **CNNL** (Cambricon Neural Network Library) for optimized operators
- **CNRTK** (Cambricon Runtime) for device management and memory allocation
- **Custom trainer** based on the ESM-2 reference implementation

# 3 Training Pipeline Architecture

## 3.1 Data Pipeline

The training data pipeline consists of three stages: (1) FASTA parsing and sequence deduplication, (2) tokenization using a 31-token amino acid vocabulary (20 standard amino acids + 5 special tokens: `[MASK]`, `[PAD]`, `[CLS]`, `[SEP]`, `[UNK]`), and (3) dynamic batching to maximize tensor utilization.

Sequences are grouped by length into buckets with stride 64 (e.g., $[1, 64], [65, 128], \ldots, [449, 512]$). Within each bucket, sequences are batched to the maximum capacity (32 sequences for length $\leq 512$). Sequences exceeding 512 tokens are truncated at the C-terminus. The resulting tensor shape per batch is $(32, 512)$ with zero-padding for shorter sequences; the attention mask distinguishes valid from padded positions.

## 3.2 Model Architecture

ESM2-small follows the ESM-2 "mini" configuration [**?** ] with 12 Transformer encoder layers, hidden dimension 256, 8 attention heads, and FFN dimension 1024. The model has 9,624,607 total parameters. Weights are stored in FP32 for optimizer state and in BF16 for forward/backward compute, consistent with MLU370's BF16 tensorcore support.

Key architectural details:

- Pre-norm residual connections (LayerNorm before attention/FFN)
- GELU activation [**?** ]
- 15% uniform token masking during MLM pre-training
- No dropout (following ESM-2 convention for fine-tuning-free evaluation)

## 3.3 Optimizer and Learning Rate Schedule

We use AdamW [**?** ] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$, and weight decay $0.01$. The learning rate schedule consists of:

1. **Linear warmup**: 1,000 steps from 0 to $1e-4$
2. **Cosine decay**: Anneal from $1e-4$ to $\sim 1e-9$ over the remaining steps

## 3.4 Checkpointing Strategy

We implement two checkpointing mechanisms:

- **Periodic step checkpoints**: Every 2,000 steps, we save a full model snapshot (optimizer state + model weights + lr scheduler state + RNG state). This enables exact resume from any checkpointed step.

- **End-of-epoch checkpoints**: After each epoch completes, a checkpoint is saved. Additionally, if the validation loss is lower than all previous checkpoints, a $\star_best.ptcopyiscreated$.

Each checkpoint is approximately 115 MB (FP32 model + optimizer states). We use a shared filesystem mount for checkpoint I/O to enable fault tolerance in multi-worker settings.

# 4 Training Dynamics and Measurements

## 4.1 Step-by-Step Breakdown

Each training step consists of:

1. **Data loading** (offloaded to CPU → MLU transfer via `cn AsyncMemcpy`)
2. **Forward pass** (MLM logits computation, 12 Transformer layers)
3. **MLM loss** (cross-entropy on masked positions, computed via $\texttt{cnnl}_{loss})\textbf{Backward pass}(gradientcomputati$
4. **Optimizer step** (AdamW update via `cn AdamW`)
5. **Learning rate update** (linear warmup → cosine schedule)

## 4.2 Throughput and Scalability

Table 1 summarizes the measured training throughput and resource utilization.

Table 1: Training System Metrics

| Metric | Value |
|---|---|
| Throughput (tokens/sec) | ∼30,000 |
| Batch size (sequences) | 32 |
| Sequence length | 512 |
| Tokens per step | 16,384 ($32 \times 512$) |
| Steps per epoch | ∼13,500 |
| Total steps (5 epochs) | 67,500 |
| Total tokens processed | ∼1.1 billion |
| Training time (total) | ∼11 hours |
| Average GPU memory utilized | ∼12 GB / 16 GB |
| MLU370 utilization | ∼85% |

The peak throughput of 30,000 tokens/second corresponds to approximately 1.8 steps/second. Accounting for periodic validation (every 500 steps) and checkpointing overhead, the effective training time per epoch is approximately 2 hours.

## 4.3 Gradient Statistics

We monitor gradient norms throughout training as a proxy for training stability. The gradient norm stays below 1.0 for the majority of training, with occasional spikes during the warmup phase (steps 0–1,000) where the learning rate ramps up rapidly. No gradient explosion or NaN events occurred during the full 67,500 steps.

# 5 Fault Tolerance and Recovery

## 5.1 Checkpoint Resume

We validate the checkpoint mechanism by simulating an interruption at step 38,000 and resuming from the nearest checkpoint (`checkpoint_step36000_best.pt`). The resumed run:

1. Correctly loads model weights, optimizer state, and RNG seed
2. Resumes the learning rate schedule from the exact step

3. Produces numerically identical loss values on the first 100 post-resume steps

This confirms the checkpoint system is sound for production use.

## 6 Related Systems Work

Training large models requires careful systems engineering. GPipe [**?** ] and Megatron-LM [**?** ] established pipeline and tensor parallelism strategies for GPU clusters. For protein models specifically, the ESM-2 training infrastructure [**?** ] was built on GPU (V100/A100); our work adapts these principles to the Cambricon MLU370 ecosystem. The MLU370 programming guide and CNNL documentation provide the baseline operator implementations we build upon.

## 7 Discussion

Training ESM2-small on MLU370 demonstrates that non-GPU accelerators are viable for protein language model pre-training, albeit with lower absolute throughput than top-tier GPUs. The MLU370's BF16 support and on-chip SRAM reduce memory pressure for models in the 10M–100M parameter range, making it suitable for research-scale experiments.

Several improvements are possible. First, multi-card data parallelism via MLULink gradient all-reduce could reduce epoch time proportionally ($N$ cards $\rightarrow 1/N$ time). Second, activation checkpointing (gradient checkpointing) would trade compute for memory, enabling larger batch sizes. Third, operator fusion (e.g., fused QKV attention + projection) could improve the effective compute-to-memory ratio.

## 8 Conclusions

We presented a systems-level account of training a 9.6M-parameter protein language model on the Cambricon MLU370 accelerator. Key findings: (1) MLU370 sustains 30K tokens/sec for Transformer MLM workloads; (2) the training is stable across 67,500 steps with no divergence; (3) checkpoint-based fault tolerance enables reliable resume from interruption. The full training codebase, data pipeline, and model weights are publicly available.

## Reproducibility

- Code: `https://github.com/junior1p/ESM2-small`
- Model weights: `https://github.com/junior1p/ESM2-small/releases/tag/v1.0.0`
- Training data: Swiss-Prot (downloaded via `rest.uniprot.org`)
- Hardware: Cambricon MLU370 (single card)

## References


[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: pre-training of deep bidirectional transformers for language understanding. *NAACL*, 2018.

[2] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelu). *arXiv*, 2020.

[3] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Chen, Mingxing Chen, Hsiang-Fu Lee, Jiquan Ngiam, Qiang Guo, Jingjing Wu, et al. Gpipe: efficient training of giant neural networks using pipeline parallelism. *NeurIPS*, 2019.

[4] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *ICLR*, 2017.

[6] Mohammad Shoeybi, Mostafa Patwary, Raul Puri, Patrick LeGresley, and Bryan Catanzaro. Megatron-lm: training multi-billion parameter language models using model parallelism. *arXiv*, 2019.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.